

P2226R0

A proposal for an idiom to move from an object and reset it to its default constructed state

Published Proposal, 2020-10-02

Issue Tracking:

[Inline In Spec](#)

Author:

[Giuseppe D'Angelo](#)

Audience:

SG18, SG1, SG20

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes two new CPOs called `take` and `take_assign`; they reset an object to its default constructed state, and return the old value of that object or assign the old value to another object. Effectively, `T t = take(obj)` should become the idiomatic form for C++14's `T t = exchange(obj, {})`; and `take_assign(new_obj, old_obj)` the idiomatic form for `new_obj = exchange(old_obj, {})`. Formally, the new value of `obj` is equal to the one achieved through value initialization, not default initialization (e.g. `take` called on an object of pointer type will reset it to `nullptr`; cf. [dcl.init.general]). In the paper we are however going to liberally talk about "default constructed" state or value, implying the state (resp. value) reached through value initialization.

Table of Contents

- 1 **Changelog**
- 2 **Tony Tables**
- 3 **Motivation and Scope**
 - 3.1 Streamlining `exchange`
 - 3.2 *Safe* move semantics
 - 3.3 Customizing `take`
 - 3.4 The need for `take_assign`
- 4 **Impact On The Standard**
- 5 **Design Decisions**
 - 5.1 Bikeshedding: naming
 - 5.2 Why not simply defaulting the second parameter of `exchange` to `T()`?
 - 5.3 What kind of exception safety should `take` offer?
 - 5.4 Should there be `take`-based algorithms, iterators, etc.?
 - 5.5 Do we need member `take` / `take_assign` for some Standard Library classes, like the member `swap`?
 - 5.6 What are the post-conditions that an overload of `take` / `take_assign` for a user-defined type must respect w.r.t. the taken-from object?
 - 5.7 Should there be an `atomic_take`?
 - 5.8 Should we promote the usage of `take` over `move` (in education materials, coding guidelines, etc.)?

- 5.9 `take` guarantees a move *and* resets the source object; on the other hand, `move` does not even guarantee a move. Should there be another function that guarantees a move, but does not reset?

6 Technical Specifications

6.1 Implementation

6.1.1 `take` and `take_assign`

6.1.2 Algorithms

6.2 Feature testing macro

6.3 Proposed wording

7 Acknowledgements

References

Informative References

Issues Index

§ 1. Changelog

- R0
 - First submission

§ 2. Tony Tables

Before	After
<pre> class C { Data *data; public: // idiomatic, C++14 C(C&& other) noexcept : data(std::exchange(other.data, {})) {} }; </pre>	<pre> class C { Data *data; public: // idiomatic, C++2? C(C&& other) noexcept : da }; </pre>
<pre> template <typename K, typename V, template <typename...> typename C = std::vector> class flat_map { C<K> m_keys; C<V> m_values; public: /* * If flat_map wants to ensure a "valid but not specified" * state after a move, then it cannot default its move * operations: C's own move operations may leave flat_map * in an invalid state, e.g. with m_keys.size() != m_values.size() * (breaking a more than reasonable class invariant for flat_map). * In other words, "valid but not specified" states do not compose; * we need to reset them to a fully specified state to restore * this class' invariants. */ flat_map(flat_map&& other) noexcept(~~~) : m_keys(std::exchange(other.m_keys, {})), m_values(std::exchange(other.m_values, {})) {} flat_map &operator=(flat_map&& other) noexcept(~~~) { m_keys = std::exchange(other.m_keys, {}); m_values = std::exchange(other.m_values, {}); return *this; } }; </pre>	<pre> template <typename K, typename V, template <typename...> typename C = std::vector> class flat_map { C<K> m_keys; C<V> m_values; public: // same, idiomatic flat_map(flat_map&& other) : m_keys(std::take_oth m_values(std::take_o {} flat_map &operator=(flat_m { std::take_assign(m_key std::take_assign(m_val return *this; } }; </pre>
<pre> void Engine::processAll() { // process only the data available at this point for (auto& value : std::exchange(m_data, {})) { // may end up writing into m_data; we will not process it, // and it will not invalidate our iterators processOne(value); } } </pre>	<pre> void Engine::processAll() { // same, idiomatic for (auto& value : std::ta processOne(value); } } </pre>
<pre> void ConsumerThread::process() { // grab the pending data under mutex protection, // so this thread can then process it Data pendingData = [&]() { std::scoped_lock lock(m_mutex); return std::exchange(m_data, {}); }(); } </pre>	<pre> void ConsumerThread::process() { Data pendingData = [&]() { std::scoped_lock lock(return std::take/m_dat }(); } </pre>

Before	After
<pre> for (auto& value : pendingData) process(value); } </pre>	<pre> for (auto& value : pending process(value); } </pre>
<pre> void Engine::maybeRunOnce() { if (std::exchange(m_shouldRun, false)) run(); } </pre>	<pre> void Engine::maybeRunOnce() { if (std::take(m_shouldRun) run(); } </pre>
<pre> struct S { // unconditional sinks // overloaded for efficiency following F.15 void set_data(const Data& d); void set_data(Data&& d); } s; Data d = ~~~; // sink "d", but leave it in a *specified* valid state s.set_data(std::exchange(d, {})); assert(d == Data()); </pre>	<pre> struct S { void set_data(const Data& void set_data(Data&& d); } s; Data d = ~~~; // same, but idiomatic s.set_data(std::take(d)); assert(d == Data()); </pre>

In the example we are referring to the [C++ Core Guideline F.15](#).

§ 3. Motivation and Scope

§ 3.1. Streamlining `exchange`

C++14, with the adoption of [\[N3668\]](#), introduced the `exchange` utility function template. `exchange` is commonly found in the implementation of move operations, in algorithms, and in other similar scenarios. Its intent is to streamline multiple operations in one function call, making them less error prone, and ultimately creating an idiom:

```

struct MyPtr {
    Data *d;

    // BAD, the split makes it possible to forget to reset other.d to nullptr
    MyPtr(MyPtr&& other) : d(other.d) { other.d = nullptr; }

    // BETTER, use std::exchange
    MyPtr(MyPtr&& other) : d(std::exchange(other.d, nullptr)) {}

    // GOOD, idiomatic: use std::exchange, generalizing
    MyPtr(MyPtr&& other) : d(std::exchange(other.d, {})) {}

    void reset(Data *newData = nullptr)
    {
        // BAD, poor readability
        swap(d, newData);
        if (newData)
            dispose(newData);

        // BETTER, readable
        Data *old = d;
        d = newData;
        if (old)
            dispose(old);
    }
}

```

```

// GOOD, streamlined
if (Data *old = std::exchange(d, newData))
    dispose(old);
}
};

```

By surveying various code bases, we noticed a common pattern: **a significant amount (50%-90%) of calls to `exchange` uses a default constructed value as the second parameter**. The typical call has the idiomatic form `exchange(obj, {})`, or it has some other form that could still be rewritten into that one (like `exchange(pointer, nullptr)` or `exchange(boolean_flag, false)`).

For instance, here's some results from very popular C++ projects:

Project	Number of calls to <code>exchange</code>	Number of calls to <code>exchange(obj, {})</code> or equivalent (i.e. calls that could be replaced by <code>take</code>)	Percentage	Notes
Boost 1.74.0	121	97	80%	Incl. calls to <code>boost::exchange</code> , as well as <code>boost::exchange</code> 's own autotests.
Qt (qtbases and qtdeclarative repositories, dev branch)	37	33	89%	Incl. calls to <code>qExchange</code> . Of the 4 calls that do not use a default constructed second argument, 2 are actually workaround for broken/legacy APIs and may get removed in the future.
Absl (master branch)	10	9	90%	Incl. calls to <code>absl::exchange</code> ; the 1 call that cannot be replaced comes from <code>absl::exchange</code> 's own autotests.
Firefox (mozilla-central repository)	14	10	71%	Incl. calls to <code>skstd::exchange</code> .
Chromium (master branch)	38	30	79%	

Note: it is interesting that, one way or another, several projects introduced their own version of `exchange` in order to be able to use it without a C++14 toolchain.

The observation of such a widespread pattern led to the present proposal. Obviously, the figures above do not include any code path where the semantic equivalent of `exchange` is "unrolled" in those codebases; nonetheless, **we claim that there is positive value for the C++ community if the pattern of "move out the old value, set a new default constructed one" could be given a name, and become an idiom on its own**. If the chosen name for such a pattern is clear and sufficiently concise, it would improve the usage of `exchange(obj, {})` (which is heavy on the eyes and somehow distracts from the actual intentions).

We propose to call this idiom the **take** idiom. This paper introduces two function objects, `take` and `take_assign`, that can be used to streamline the calls to `std::exchange` (depending on whether we are constructing a new object or assigning onto an existing one).

§ 3.2. Safe move semantics

We also believe that such an idiom would become a useful tool in understanding and using move semantics. `take`, as presented, can be used as a drop-in replacement for `move` (under the reasonable assumption that movable types are also,

typically, default constructible). Unlike `move`, it would leave the source object in a well-defined state -- its default constructed state:

```
f(std::move(obj)); // obj's state is unknown
                  // - could be valid but unspecified
                  // - could be not valid / moved from
                  // - potentially, could be not moved-from at all...

// VERSUS

f(std::take(obj)); // obj has specified state
```

Using [Sean Parent's](#) definitions, `take` would constitute a *safe* operation, and be the counterpart of `move` (an *unsafe* operation).

As for some other differences between the two:

	move	take
Does <code>auto new_obj = xxxx(old_obj); throw</code> exceptions?	Usually no: move constructors are commonly <code>noexcept</code>	Depends: generally no assumptions can be made regarding the default constructor
What is the state of <code>old_obj</code> after the call above?	Usually <i>unspecified</i> ; depending on the class, it's valid or not valid/partially formed	<i>Specified</i> : default constructed state
If a type is cheap to move, is the above call cheap?	Yes (tautological)	Depends: generally no assumptions can be made regarding the default constructor
What does <code>obj = xxxx(obj); do?</code>	Leaves <code>obj</code> in its moved-from state, or leaves <code>obj</code> in its original state, depending on the implementation	Leaves <code>obj</code> in its original state, assuming no exceptions occur; it's, however, expensive and not a no-op!
If I no longer need <code>old_obj</code> , is <code>new_obj = xxxx(old_obj); a good idea?</code>	Yes	No

In conclusion: we believe the Standard Library should offer both `move` and `take` to users; each one has its merits and valid use cases.

§ 3.3. Customizing `take`

`take(obj)` can be implemented just as `exchange(obj, {})`, that is:

```
1 template <class T>
2 constexpr T take(T& obj)
3 {
4     T old_val = move(obj);
5     obj = T();
6     return old_val;
7 }
```

While this is a valid general-purpose implementation, it's not necessarily the most efficient for all `T`s. For instance, what if the move constructor of a given type leaves the moved-from object already in the default constructed state? The reset on line 5 would be not necessary.

One can build other similar examples where the generic code would be suboptimal:

- in case of types implemented out-of-line the generic code places `N` out-of-line calls (move constructor, default constructor, move assignment, destructor, plus the return mechanism), but it could be replaced by just one out-of-line call;

- for wrapper types like `std::array` one doesn't need a temporary, default-constructed one; instead, `take` could be applied to the individual components.

There is a strong analogy with `swap` here. `swap` is offered as a customization point because a type's author could implement it better than what the Standard Library can do in its general-purpose implementation. It's very likely that a type that features a custom `swap` could also benefit from a better version of `take` than the one above.

Therefore, we claim that `take` should also be a customization point. The implementation above can be the default provided by the Standard Library; but type authors can provide their own version if it makes sense for their own types. For this to work, we propose making `take` a CPO, so that it would automatically find suitable overloads of `take` for user-defined datatypes.

§ 3.4. The need for `take_assign`

Using `take` to assign over an existing object may result in suboptimal code which cannot be improved even with a customized overload for `take`.

Let's consider an expression like:

```
another_obj = take(obj);
```

This expression requires the construction and destruction of a temporary object (the return value from `take`) that cannot be elided or removed. This prevents this code to be optimal -- the same considerations regarding the customization of `take` apply here.

For instance, for a type whose move operations leave the moved-from object in the default constructed state, then the above could be rewritten in a plain move assignment (ignoring the issue of self-moves):

```
// this could be equivalent to take for some types
another_obj = std::move(old_obj);
```

This will have the same effects, but *without* the intermediate temporary object. As a consequence, we cannot propose `take` for this use-case (or it won't get used in practice, playing the "poor performance" card).

The fundamental advantage of `move` over `take` in the above example is that the move assignment operator has access to *both* the assigned-to and the assigned-from objects. Such advantage is unachievable by a function like `take`, which knows nothing about the assigned-to object (it doesn't even know that it is being used in an assignment). **The conclusion is that in order to be able to offer the same level of performance we need a function that implements take-assignment.**

We propose `take_assign` as the name of that function; just like `take`, it should be a customization point.

§ 4. Impact On The Standard

This proposal is a pure library extension.

It proposes changes to existing headers, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables.

This proposal does not require any changes in the core language.

This proposal does not depend on any other library extensions.

§ 5. Design Decisions

The most natural place to add the `take` and `take_assign` CPOs presented by this proposal is the already existing `<utility>` header, following the precedent of `move` and `exchange`.

§ 5.1. Bikeshedding: naming

We foresee that finding the right name for the proposed idiom / function objects is going to be a huge contention point. Therefore, we want to kickstart a possible discussion right away. In R0, we are proposing `take`, inspired by Rust's own `std::mem::take` function, which has comparable semantics to the ones defined here.

Other possible alternatives include:

- `reset`
- `move_and_reset`
- `safe_move` (modelled after [Sean Parent](#)'s terminology)
- `transfer`
- `collect`
- `grab`

We strongly believe that this idiom needs a concise name in order to be useful; therefore we are not proposing something like `exchange_with_default_constructed`.

ISSUE 1 Submit a poll to LEWG(I), seeking ideas and consensus for a name.

§ 5.2. Why not simply defaulting the second parameter of `exchange` to `T()`?

[\[N3668\]](#) mentions this idea, but indeed rejects it because it makes the name `exchange` much less clear:

```
another_obj = std::exchange(obj); // exchange obj... with what? with another_obj? wait, is
```

We agree with that reasoning, so we are not proposing to change `exchange`. Moreover, this would also mean making `exchange` a customization point to reap the same benefits of a custom `take`, which is a breaking change at this point.

§ 5.3. What kind of exception safety should `take` offer?

The same of `exchange`, namely, the basic exception safety. Obtaining a higher level is out of scope and not achievable without pessimizing the common use case.

§ 5.4. Should there be `take`-based algorithms, iterators, etc.?

Yes, and we are proposing them.

§ 5.5. Do we need member `take` / `take_assign` for some Standard Library classes, like the member `swap`?

We are unable at this moment at giving a definitive answer; the motivations behind making `swap` a member are unclear to the author -- is it convenience or part of the customization design of `swap`, which maybe is not necessary when using CPOs?

ISSUE 2 We need to seek LEWG(I) guidance here.

§ 5.6. What are the post-conditions that an overload of `take` / `take_assign` for a user-defined type must respect w.r.t. the taken-from object?

By making `take` a customization point, it becomes essential to clearly document what user-provided overloads are supposed to do. In particular it might not be what "reset to the default constructed" state really entails.

Let's consider `std::string`, used like this:

```
std::string s, t = ~~~;

// hand-rolled take_assign
t = std::move(s);
s = std::string();
```

It's clear that, after this code, `s` is in a valid state, and it's empty -- matching the default-constructed state of a `std::string` object. But `s` may not be *fully* in its default-constructed state: for instance, it may be carrying extra capacity (which actually happens on [libstdc++](#)).

```
assert(s.empty());           // OK
assert(s == std::string()); // OK
assert(s.capacity() == std::string().capacity()); // ERROR
```

This is fine: the capacity of a default constructed `std::string` is unspecified, so one cannot assume anything about it.

The same line of reasoning should be applied to `take`. Summarizing, a taken-from object:

1. must have the same *documented* state and invariants of a default constructed object of its type;
2. has any state which is unspecified on default construction reset to unspecified (even if it was specified before the call to `take`);
3. if its type models `equality_comparable`, it must compare equal to a default constructed object.

In case of `take_assign`, the above must be guaranteed even in case of a "self-take".

§ 5.7. Should there be an `atomic_take`?

The rationale of adding `exchange` (with that specific name) in the Standard Library was generalizing the already existing semantics of `atomic_exchange`, extending them to non-atomic, movable types. By having `take(obj)` as a "shortcut" for `exchange(obj, {})`, one may indeed wonder if also `atomic_take(obj)` should be added, as a shortcut for `atomic_exchange(obj, {})` (mut. mut. for `atomic<T>::take`).

We are not fully convinced by the usefulness of `atomic_take` and therefore we are not proposing it here. First and foremost, unlike `exchange`, `atomic_exchange` has many more uses where the second argument is *not* a default constructed value. Second, the overwhelming majority of usage of atomic types consist of atomic integral types and atomic pointer types. We do not believe that substituting `atomic_exchange(val, 0)` or `atomic_exchange(val, nullptr)` with `atomic_take(val)` would convey the same "meaning" when used in the context of atomic programming.

We would be happy to be convinced otherwise.

ISSUE 3 Ask SG1 for their opinion regarding `atomic_take`.

§ 5.8. Should we promote the usage of `take` over `move` (in education materials, coding guidelines, etc.)?

No. We believe we need instead to educate users about the availability of both options in the Standard Library, make them understand the implications of each one, and let the users choose the right tool for the job at hand. To give an example: it sounds extremely unlikely that one should be using `take` inside a shuffle-based algorithm (the impact in performance and correctness would be disastrous when compared to using `move` instead).

There is however an important point to be made: the state of a moved-from object has been actively disputed in the C++ community pretty much since the very introduction of move semantics. The usual position is one between these two:

1. object is valid but unspecified (e.g. [lib.types.movedfrom], [Sutter], [C.64]), and therefore it can be used in any way that does not have preconditions (or similarly it would still be possible to check such preconditions before usage); or
2. object is partially formed / not valid, (e.g. [P2027R0]), and therefore the only operations allowed on such an object are assignment and destruction.

This debate has not yet reached a conclusion. In the meanwhile, *what should a user handling a generic object of type `T` do, if they want to keep using it after it has been moved?* The simplest solution would be to reset the object to a well known state. If there is already such a well known state readily available for the user, then the combination of `move` + "reset to state X" (however that would be expressed in code) makes perfect sense and it's certainly the way to go. Otherwise, the easiest state to reason about objects of type `T` is their default constructed state; therefore one may want to reset their moved-from object to such default constructed state, and then keep using the object. Moving plus resetting to default constructed state is precisely what `take` does.

We would like to clarify that we do not have any sort of "hidden agenda" that wants to settle the state of a moved-from object (in the Standard Library, or in general). And, we absolutely do not claim that moved-from objects should always be reset to their default constructed state (in their move operations, or by always using `take` instead of `move` in one's codebase). **The availability of `take` for users should simply constitute another tool in their toolbox, allowing them to choose which kind of operation (safe/unsafe) makes most sense for their programs at any given point.**

§ 5.9. `take` guarantees a move *and* resets the source object; on the other hand, `move` does not even guarantee a move. Should there be another function that guarantees a move, but does not reset?

In other words, should there be some sort of "intermediate" function between `move` and `take`, that simply guarantees that the input object will be moved from? For instance:

```
template <class T>
constexpr T really_move(T& obj) requires (!is_const_v<T>) /* or equivalent */
{
    T moved = move(obj);
    return moved;
}
```

A possible use case would be to "sink" an object, therefore deterministically releasing its resources from the caller, even if the called function does not actually move from it:

```
template <class Fun>
void really_sink(Fun f)
{
    Object obj = ~~~;

    f(std::move(obj)); // if f doesn't actually move,
                      // we still have obj's resources here in the caller

    // VERSUS

    f(std::really_move(obj)); // obj is moved from, always.
                             // using take would be an overkill
                             // (for instance, obj is not used afterwards,
                             // so why resetting it?)
}
```

Now, having functions which have parameters of type rvalue reference (or forwarding reference), and then do not `move` / `forward` them unconditionally in all code paths, is generally frowned upon (cf. [F.18] and [F.19], and especially the "Enforcement" sections). The possibility for `move` to actually *not* move seems more a theoretical exercise than an issue commonly found in practice.

In any case, we do not have enough data to claim that there is a "widespread need" for such a function in the Standard Library; surveying the same projects listed in § 3 Motivation and Scope gives inconcludent results (it seems that such a function is not defined in any of them, for their own internal purposes).

Therefore, we are not going to propose such a `really_move` function here.

We do not think that `take` does impede in any way the addition of such a function anyhow, via a separate proposal. One might even argue that the addition of such a `really_move` function should be not tied to the addition of `take`, as it solves a different problem: ensuring that an object gets always moved from.

§ 6. Technical Specifications

§ 6.1. Implementation

§ 6.1.1. `take` and `take_assign`

`take` and `take_assign` are CPOs that needs to dispatch to ADL-found `take` (resp. `take_assign`), if found, or fall back to a default implementation provided by the Standard Library.

The actual design (finding members, finding non-members, etc.) needs some feedback depending on the answers for the issue raised at [§ 5.5 Do we need member `take` / `take\_assign` for some Standard Library classes, like the member `swap`?](#).

The default implementation should be straightforward, for instance like this:

```
template <class T>
constexpr T take(T& obj)
{
    T old_val = move(obj);
    obj = T();
    return old_val;
}

template <class T>
constexpr void take_assign(T& target, T& source)
{
    target = move(source);
    source = T();
}
```

§ 6.1.2. Algorithms

Add:

- `take`, `take_backward` (in `<algorithm>` and `<ranges>`)
- `uninitialized_take`, `uninitialized_take_n` (in `<memory>`)
- `take_iterator`, `make_take_iterator`, `take_sentinel` (in `<iterator>`)

§ 6.2. Feature testing macro

We propose `__cpp_lib_take`.

§ 6.3. Proposed wording

All changes are relative to [\[N4861\]](#).

TBD.

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Marc Mutz for reviewing this proposal, and pointing me to [Sean Parent's](#) blog post. His educational work regarding move semantics has been inspirational. He originally proposed the idea of an idiomatic form for `std::exchange(obj, {})` on the [std-proposals mailing list](#).

Thanks to Arthur O’Dwyer for the early feedback.

All remaining errors are ours and ours only.

§ References

§ Informative References

[C.64]

Bjarne Stroustrup; Herb Sutter. [C++ Core Guidelines, C.64: A move operation should move and leave its source in a valid state](https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-move-semantic). URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-move-semantic>

[F.15]

Bjarne Stroustrup; Herb Sutter. [C++ Core Guidelines, F.15: Prefer simple and conventional ways of passing information](https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-conventional). URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-conventional>

[F.18]

Bjarne Stroustrup; Herb Sutter. [C++ Core Guidelines, F.18: For “will-move-from” parameters, pass by X&& and std::move the parameter](https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f18-for-will-move-from-parameters-pass-by-x-and-stdmove-the-parameter). URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f18-for-will-move-from-parameters-pass-by-x-and-stdmove-the-parameter>

[F.19]

Bjarne Stroustrup; Herb Sutter. [C++ Core Guidelines, F.19: For “forward” parameters, pass by TP&& and only std::forward the parameter](https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f19-for-forward-parameters-pass-by-tp-and-only-stdforward-the-parameter). URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f19-for-forward-parameters-pass-by-tp-and-only-stdforward-the-parameter>

[MarcMutz]

Marc Mutz. [Is `std::exchange\(obj, {}\)` common enough to be spelled `std::move\_and\_reset\(obj\)`?](https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/qDB0BG-GQqQ/discussion). URL: <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/qDB0BG-GQqQ/discussion>

[N3668]

Jeffrey Yasskin. [exchange\(\) utility function, revision 3](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3668.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3668.html>

[N4861]

Richard Smith, Thomas Koeppel, Jens Maurer, Dawn Perchik. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2020/n4861.pdf). URL: <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2020/n4861.pdf>

[P2027R0]

Geoff Romer. [Moved-from objects need not be valid](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2027r0.pdf). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2027r0.pdf>

[SeanParent]

Sean Parent. [About Move](https://sean-parent.stlab.cc/2014/05/30/about-move.html). URL: <https://sean-parent.stlab.cc/2014/05/30/about-move.html>

[StdMemTake]

Rust. [Function `std::mem::take`](https://doc.rust-lang.org/std/mem/fn.take.html). URL: <https://doc.rust-lang.org/std/mem/fn.take.html>

[StdStringCapacityAfterMoveAssignment]

[std::string retains capacity after move assignment](https://godbolt.org/z/6rfPYv). URL: <https://godbolt.org/z/6rfPYv>

[Sutter]

Herb Sutter. [Move, simply](https://herbsutter.com/2020/02/17/move-simply/). URL: <https://herbsutter.com/2020/02/17/move-simply/>

§ Issues Index

ISSUE 1 Submit a poll to LEWG(I), seeking ideas and consensus for a name. [↩](#)

ISSUE 2 We need to seek LEWG(I) guidance here. [↩](#)

ISSUE 3 Ask SG1 for their opinion regarding `atomic_take`. [↩](#)

ISSUE 4 Poll LEWG(I) for more opinions. [↩](#)